

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 4

Выполнил:

Баженов Алексей

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашнего задания требовалось разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service

Спроектировать микросервисную архитектуру

Описать способы их взаимодействия друг с другом

Спроектировать разделение БД на обособленные БД

Спроектировать и описать эндпоинты, необходимые для межсервисного взаимодействия в формате OpenAPI

Описать варианты запросов и ответов, задокументировать возможные ошибки

В результате должен сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения монолитного бэкенд-приложения на микросервисы

Ход работы

В рамках выполнения домашнего задания был проведён анализ монолитного бэкенд-приложения системы бронирования ресторанов, реализованного на Node.js/Express с использованием TypeORM и СУБД PostgreSQL

Этап 1. Анализ текущего монолита

Приложение, включало в себя:

Контроллеры (5 модулей):

- аутентификация, логин, JWT
- управление пользователями
- рестораны, столики, меню, фото
- бронирования, отмена
- отзывы и рейтинги

Моделли (7 сущностей):

пользователи, рестораны, столики, позиции меню, фотографии, бронирования, отзывы

Вспомогательные модули:

- JWT верификация
- автоматическое хэширование паролей
- bcrypt утилиты

Инфраструктура:

- единая PostgreSQL БД со всеми 7 таблицами
- Docker Compose для БД
- Swagger для документации

Выявлены ключевые проблемы монолитной архитектуры:

- единая точка отказа
- невозможность независимого масштабирования разных доменов
- связанность кодовой базы
- отсутствие изоляции данных

Этап 2. Проектирование микросервисной архитектуры

Для каждого bounded context спроектирован отдельный микросервис с собственной базой данных, вот тут как раз и придерживание концепции database-per-service

Декомпозиция на микросервисы

На основе анализа предметной области и структуры базы данных выделены четыре микросервиса, каждый из которых владеет собственным набором таблиц и предоставляет изолированный функционал.

Таблица 1 — Состав микросервисов

Микросервис	Порт	База данных	Таблицы	Функционал
auth-user-service	8001	auth_db	users	Регистрация, логин, JWT, профиль

				пользователя
restaurant-service	8002	restaurant_db	restaurants, tables, menu_items, restaurant_photos	Управление каталогом ресторанов, столиками, меню, фотографиями, поиск и фильтрация
reservation-service	8003	reservation_db	reservations	Создание и отмена бронирований, проверка доступности столиков
review-service	8004	review_db	reviews	Отзывы, рейтинги

Этап 3. Атрибуты и связи внутри микросервисов

auth-user-service (база auth_user_db)

Содержит единственную таблицу Users с полями для аутентификации и профиля:

Поле	Тип	Описание
id	int PK	Идентификатор
fullName	string	Полное имя
email	string (unique)	Email для входа
password	string	Хэш пароля (bcrypt)

Связей с другими таблицами внутри сервиса нет.

restaurant-service (база restaurant_db)

Содержит четыре таблицы, образующие каталог ресторанов:

Таблица	Связь	Описание
restaurants	центральная	Рестораны с атрибутами (название, адрес, город, кухня, ценовая категория)
tables	многие-к-одному к restaurants	Столики (вместимость, расположение, статус)
menu_items	многие-к-одному к restaurants	Блюда меню (название, цена, категория)
restaurant_photos	многие-к-одному к restaurants	Фотографии ресторанов (URL, подпись)

Все связи внутри сервиса — «один-ко-многим» от restaurants к зависимым таблицам.

reservation-service (база reservation_db)

Содержит одну таблицу reservations:

Поле	Тип	Описание
id	int PK	Идентификатор брони
userId	int (логический FK)	Ссылка на пользователя в auth_user_db
tableId	int (логический FK)	Ссылка на столик в restaurant_db
reservationDate	date	Дата бронирования
startTime	string	Время начала
endTime	string	Время окончания
guestsCount	int	Количество гостей
status	string	Статус (confirmed, cancelled)
createdAt	date	Дата создания

review-service (база review_db)

Содержит одну таблицу reviews:

Поле	Тип	Описание
id	int PK	Идентификатор отзыва
userId	int (логический FK)	Ссылка на пользователя в auth_user_db
restaurantId	int (логический FK)	Ссылка на ресторан в restaurant_db
rating	int	Оценка (1-5)
comment	string	Текст отзыва
createdAt	date	Дата создания

Этап 4. Межсервисное взаимодействие

Взаимодействие между микросервисами осуществляется синхронно через HTTP REST. Каждый сервис предоставляет внутренние эндпоинты, доступные только внутри сети микросервисов.

Таблица 2 — Межсервисные связи

От сервиса	К сервису	Эндпоинт	Назначение
reservation-service	auth-user-service	GET /internal/v1/users/{id}/exists	Проверка существования пользователя при создании брони
reservation-service	restaurant-service	GET /internal/v1/tables/{id}/validate	Проверка столика (существует, вместимость, свободен ли)
review-service	restaurant-service	POST /internal/v1/restaurants/{id}/update-rating	Обновление рейтинга ресторана после нового отзыва

Обратное взаимодействие отсутствует — сервисы не создают циклических зависимостей.

Этап 5. Спецификация внутренних эндпоинтов (OpenAPI)

5.1 auth-user-service Internal API (порт 8001)

openapi: 3.0.0

info:

title: Auth User Service Internal API

version: 1.0.0

paths:

/internal/v1/users/{userId}/exists:

get:

summary: Проверить существование пользователя

parameters:

- name: userId

in: path

required: **true**

schema:

type: integer

responses:

200:

description: Пользователь существует

content:

application/json:

schema:

type: object

properties:

exists:

type: boolean

user:

type: object

properties:

id:

```
      type: integer
    email:
      type: string
    fullName:
      type: string
  404:
    description: Пользователь не найден
    content:
      application/json:
        schema:
          type: object
          properties:
            error:
              type: string
            code:
              type: string
            enum: [USER_NOT_FOUND]
```

5.2 restaurant-service Internal API (порт 8002)

```
paths:
  /internal/v1/tables/{tableId}/validate:
    get:
      summary: Проверить столик для бронирования
      parameters:
        - name: tableId
          in: path
          required: true
          schema:
            type: integer
        - name: date
          in: query
```

required: **true**

schema:

type: string

format: date

- name: startTime

in: query

required: **true**

schema:

type: string

- name: endTime

in: query

required: **true**

schema:

type: string

responses:

200:

description: Столик валиден

content:

application/json:

schema:

type: object

properties:

exists:

type: boolean

capacity:

type: integer

isAvailable:

type: boolean

404:

description: Столик не найден

content:

application/json:

schema:

type: object

properties:

error:

type: string

code:

type: string

enum: [TABLE_NOT_FOUND]

409:

description: Столик уже забронирован

content:

application/json:

schema:

type: object

properties:

error:

type: string

code:

type: string

enum: [TABLE_BUSY]

conflictingReservationId:

type: integer

/internal/v1/restaurants/{restaurantId}/update-rating:

post:

summary: Обновить рейтинг ресторана

parameters:

- name: restaurantId

in: path

required: **true**

```
    schema:
      type: integer
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            newRating:
              type: number
              format: float
            reviewsCount:
              type: integer
  responses:
    200:
      description: Рейтинг обновлён
      content:
        application/json:
          schema:
            type: object
            properties:
              success:
                type: boolean
    404:
      description: Ресторан не найден
```

Этап 7. Варианты запросов и ответов, возможные ошибки

Пример 1. Проверка пользователя (reservation → auth-user)

Запрос:

GET /internal/v1/users/123/exists

X-Internal-Token: secret-token-123

Успешный ответ (200):

```
{  
  "exists": true,  
  "user": {  
    "id": 123,  
    "email": "ivan@example.com",  
    "fullName": "Ivan Ivanov"  
  }  
}
```

Ответ при отсутствии пользователя (404):

```
{  
  "error": "User not found",  
  "code": "USER_NOT_FOUND"  
}
```

Пример 2. Проверка столика (reservation → restaurant)

Запрос:

GET /internal/v1/tables/42/validate?date=2026-06-10&startTime=18:00&endTime=20:00

X-Internal-Token: secret-token-123

Успешный ответ (200):

```
{  
  "exists": true,  
  "capacity": 4,  
  "isAvailable": true
```

```
}
```

Ответ при занятом столике (409):

```
{
```

```
  "error": "Table already booked",
```

```
  "code": "TABLE_BUSY",
```

```
  "conflictingReservationId": 101
```

```
}
```

Таблица возможных ошибок

Ошибка	HTTP код	Когда возникает	Действие вызывающего сервиса
User_not_found	404	Пользователь не найден	Вернуть клиенту ошибку создания брони
Table_not_found	404	Столик не найден	Вернуть клиенту ошибку
Table_busy	409	Столик уже забронирован	Предложить другое время
Table_capacity_exceeded	400	Гостей больше вместимости	Сообщить клиенту
Service_unavailable	503	Сервис не отвечает (таймаут 3 сек)	Retry с экспоненциальной задержкой (3 попытки)
Invalid_internal_token	401	Неверный X-Internal-Token	Отклонить запрос

Этап 8. План перехода от монолита к микросервисам (Strangler Fig)

Для миграции предлагается паттерн Strangler Fig, позволяющий постепенно вытеснять монолит без остановки работы приложения.

Шаг 1. API Gateway. Разворачивается Nginx, который изначально направляет все запросы /api/* на монолит. Клиенты продолжают работать без изменений.

Шаг 2. Выделение auth-user-service. Из монолита выносятся таблица users и эндпоинты аутентификации (/api/auth/*) и управления пользователями (/api/users/*). Gateway начинает маршрутизировать соответствующие пути на новый сервис (порт 8001). Монолит для проверки пользователей добавляет HTTP-клиент к внутреннему эндпоинту auth-user-service.

Шаг 3. Выделение restaurant-service. Переносятся все таблицы каталога (restaurants, tables, menu_items, restaurant_photos) и эндпоинты /api/restaurants/*. Gateway обновляет маршрутизацию. Оставшаяся часть монолита (бронирования и отзывы) для валидации столиков использует внутренние API restaurant-service (порт 8002).

Шаг 4. Выделение review-service. Выносятся таблица reviews и эндпоинты /api/reviews/* в отдельный сервис (порт 8004). Настраивается вызов к restaurant-service для обновления рейтинга ресторана после создания отзыва.

Шаг 5. Выделение reservation-service. Выносятся таблица reservations и эндпоинты /api/reservations/* в отдельный сервис (порт 8003). Настраиваются HTTP-вызовы к auth-user-service для проверки пользователя и к restaurant-service для проверки столика

Шаг 6. Бессмысленность монолита. После того как весь трафик перенаправлен на микросервисы, монолит полностью выводится из эксплуатации. Все четыре сервиса функционируют независимо со своими базами данных

Вывод

В результате выполнения работы была спроектирована микросервисная архитектура для системы бронирования ресторанов. Придерживались концепции database-per-service.

Архитектура включает 4 микросервисов (AuthUser, Restaurant, Reservation, Review) и API Gateway, каждый из владеет собственной базой данных. Для межсервисного взаимодействия используются синхронные HTTP-запросы через Internal REST API

Преимущества предложенной архитектуры:

Изоляция данных, то есть каждый сервис владеет своей БД, это исключает конфликты блокировок и обеспечивает независимость данных.

Независимое масштабирование, сервисы могут быть развёрнуты на разных узлах и масштабироваться по нагрузке независим.

Слабая связанность, чёткие интерфейсы взаимодействия через Internal API минимизируют зависимости между сервисами. Всего 3 внутренние связи

Поэтапная миграция, разработанный план по стратегии Strangler Fig позволяет постепенно выделять сервисы из монолита без остановки системы, минимизируя риски.

Ретрай на межсервисных вызовах предотвращают каскадные отказы. Результатом работы является спроектированный backend с документированным API